

061-并发程序

- 顺序程序设计
 - 顺序性：严格按照执行顺序
 - 封闭性：程序运行时独占资源
 - 确定性：执行结果和速度、时段无关
 - 可再现性：程序对相同数据集的执行轨迹确定
- 进程并发：进程在执行时间上重叠
 - CPU 在任意时间只有一个进程在执行

并发程序设计

```
// 原串行程序
while (true){
    input();
    process();
    output();
}
// 并发程序 1
while (true){
    input();
    send();
}
// 并发程序 2
while (true){
    receive();
    process();
    send();
}
// 并发程序 3
while (true){
    receive();
    output();
}
```

- 原串行程序：顺序执行，效率低
- 并程序：在`process()`，`output()`时可不断`input()`，提高效率
- 通信机制：处理制约关系，`send()`和`receive()`

并发程序设计的特性

- 并行性
- 共享性：多进程共享软件资源
- 交往性：并行时进程间存在制约

并发进程的制约

- 无关并发进程：一组并发进程在不同的变量集合上运行
 - Bernstein条件：进程的执行与时间无关需满足
 - $(R(A) \cap W(B)) \cup (W(A) \cap R(B)) \cup (W(A) \cap W(B)) = \emptyset$
 - $R(i)$ ：程序/语句*i*读的变量
 - $W(i)$ ：程序/语句*i*写的变量
- 交往的并发进程：并发进程间共享变量，一个进程的执行影响其他进程的结果
 - 进程间执行速度无法控制，可能会出现时间相关的错误：结果错误/永远等待
 - 机票售票问题：两个人同时买票，可能会出现超售
 - 主存管理问题：A 申请主存时不足，在进入等待队列前，B 完成了释放操作，A 永远无法获知有空闲内存
- 进程交互关系
 - 竞争关系：不同进程需同时访问同一资源，在资源使用时，另一进程需等待
 - 协作关系：进程在伙伴进程未完成前需阻塞
- 竞争关系带来的问题
 - 死锁问题：一组进程因相互等待对方持有的资源而永远阻塞
 - 饥饿问题：低优先级的进程永远无法获得资源
- 解决方案：
 - 互斥：同一资源只能被一个进程使用
 - 同步：进程基于某些条件协调活动
 - 在代码中指定 checkpoint + 条件等待
 - 使用 send/receive 进行显式消息通信

临界区

- 临界资源：互斥访问的资源
- 临界区：访问临界资源的代码段
- 临界区管理要求
 - 互斥：一次至多一个进程停留在临界区
 - 避免死锁：一个进程不能无限停留在临界区内
 - 避免饥饿：一个进程不能无限地等待进入临界区

软件实现临界区管理

LockOne 算法

```
bool inside[2] = {false, false}; // 进程是否在临界区

void process_0() {
    while (inside[1]); // 1
    inside[0] = true; // 2
    // 临界区
    inside[0] = false;
}

void process_1() {
    while (inside[0]); // 3
    inside[1] = true; // 4
    // 临界区
}
```

```
    inside[1] = false;
}
```

```
bool inside[2] = {false, false}; // 进程是否在临界区

void process_0() {
    inside[0] = true; // 2
    while (inside[1]); // 1
    // 临界区
    inside[0] = false;
}
void process_1() {
    inside[1] = true; // 4
    while (inside[0]); // 3
    // 临界区
    inside[1] = false;
}
```

- 每个进程维护自己的状态
- 问题：1/2, 3/4 不是原子操作
- 前一种按1, 3, 2, 4执行，两个进程都在临界区内
- 后一种按2, 4, 1, 3执行，产生死锁

LockTwo 算法

```
int turn = 0; // 轮到哪个进程

void process_0() {
    while (turn != 0); // 1
    // 临界区
    turn = 1; // 2
}
void process_1() {
    while (turn != 1); // 3
    // 临界区
    turn = 0; // 4
}
```

- 问题：若控制权所在的进程没有进入临界区的需求，另一进程被阻塞，产生饥饿
- 初始值偏向0

Peterson 算法

```
bool inside[2] = {false, false}; // 进程是否在临界区
int turn = 0; // 轮到哪个进程
```

```

void process_0() {
    inside[0] = true; // 表明有进入临界区的需求
    turn = 1; // 让出控制权
    while (inside[1] && turn == 1); // 等待对方退出/让出控制权
    // 临界区
    inside[0] = false; // 退出临界区
}

void process_1() {
    inside[1] = true;
    turn = 0;
    while (inside[0] && turn == 0);
    // 临界区
    inside[1] = false;
}

```

- 保证互斥：只有一个进程能进入临界区
- 解决死锁：引入`turn`变量后，如果两个进程都到达`while`，两个`while`的条件不会同时为真，避免等待
- 解决饥饿：`inside` 确保进程是否有进入临界区的需求

硬件实现临界区管理

关中断

- 进程切换由中断引起，关闭中断可避免进程切换，保证原子操作
- 缺点
 - 中断关闭时间过长，影响系统性能
 - 不应将此权限赋予用户
 - 多核系统中，关闭中断无法保证互斥

Test_and_Set

```

bool TS(bool *lock) {
    if (*lock) {
        *lock = false;
        return true;
    } else {
        return false;
    }
}

bool lock = true;
void process_0() {
    while (!TS(&lock));
    // 临界区
    lock = true;
}

```

- `TS`：硬件层面的原子指令，读取并设置锁的状态

- 效率低，可能产生饥饿

对换指令

```
bool lock = false;

Process Pi() { // i = 1, 2, ..., n
    bool keyi = true;
    do {
        SWAP(keyi, lock); // 尝试获得锁
    } while (keyi);      // 如果 keyi 为 true, 说明没抢到, 继续等待 (自旋)

    // 临界区 keyi = false
    SWAP(keyi, lock);    // 释放锁 (lock = false)
}
```

- **SWAP**: 硬件层面的原子指令，交换两个变量的值
- 效率低，可能产生饥饿

进程通信

- 直接通信：发送双方必须知道彼此
 - 发送者 -> 内核 -> 接收者
 - **send(P, msg)**: 向进程 **P** 发送消息 **msg**
 - **receive(P, msg)**: 从进程 **P** 接收消息 **msg**
- 间接通信：发送双方只需共用同一队列即可
 - 发送者 -> 内核 -> 消息队列 -> 接收者
 - **send(Q, msg)**: 向消息队列 **Q** 发送消息 **msg**
 - **receive(Q, msg)**: 从消息队列 **Q** 接收消息 **msg**

死锁

- 死锁：一组进程因相互等待对方持有的资源而永远阻塞

死锁的条件

- 互斥：系统中存在临界资源
- 占有且等待：一个进程持有资源并等待其他资源
- 不剥夺/不可抢占：资源不能被强制剥夺
- 循环等待：存在一个进程等待链，链中每个进程都在等待下一个进程持有的资源

死锁的防止

- 使资源同时访问 $\Rightarrow$ 互斥
- 静态分配 $\Rightarrow$ 占有且等待
- 进程申请不到资源时主动释放 $\Rightarrow$ 不剥夺
- 层次分配策略 $\Rightarrow$ 循环等待
 - 申请低级资源后，只能申请高级资源

- 释放低级资源前，先释放高级资源
- 申请同级资源必须先释放同级已占用的资源

银行家算法

- 核心思想：仅当剩余资源足够满足所有进程的最大需求时，才允许新进程进入系统
- 假设有 n 个进程， m 种资源类型
- 每类资源总数 $R_i = (R_1, R_2, \dots, R_m)$
- 未分配资源数 $V_i = (V_1, V_2, \dots, V_m)$
- 每个进程的最大需求 $C_{n \times m}$
- 每个进程已分配资源数 $A_{n \times m}$
- 系统安全性：发生死锁一定在不安全状态
 - 对进程进行排列，存在某个进程序列， $P_1, P_2, \dots, P_n$ ，对于每一个进程 P_k ，对于所有的资源 i ，满足： $C_{k,i} - A_{k,i} \leq \text{Available}_i$ （PPT把简单的事情搞得很复杂）
- 步骤：
 - 对于每个进程 i ，计算 $C_{i,j} - A_{i,j}$ ，得到 **Need** 矩阵
 - 选择一个进程 i ，满足 $\text{Need}_{i,j} \leq \text{Available}_j$ ，将其加入安全序列
 - 更新 **Available** 矩阵： $\text{Available}_j = \text{Available}_j + A_{i,j}$ （进程 i 执行完成，释放资源）
 - 重复上述步骤，直到所有进程都被加入安全序列或无法找到满足条件的进程

死锁检测

- 资源分配图
 - 节点：进程和资源
 - 边：分配边（从资源到进程）和请求边（从进程到资源）
- 若资源分配图无环路，则无死锁
- 若存在环路，且每个资源类只有一个实例，则存在死锁
- 若存在环路，且资源类有多个实例，则可能存在死锁
- 图的简化
 1. 找到一个出边只有请求边，且请求的资源系统有空闲的进程；
 2. 满足它的请求（即资源分配）；
 3. 假设它完成，释放资源（去掉它的分配边）；
 4. 成为孤立结点，从图中移除；
 5. 重复上述过程，看是否所有进程都能这样“被移除”。
- 若资源分配图不可完全简化，则存在死锁